STM32基础例程

- STM32基础例程
- 硬件准备
- 实验目的
- HT-10A遥控器
- SBUS接收机
- STM32工程
  - 配置引脚
    - ROS控制板原理图
    - 工程引脚配置
  - 实现原理
  - 核心代码
    - 串口功能
    - SBUS数据
  - 主函数
- 硬件连接
- 下载程序
- 实验现象

硬件准备

- HT-10A遥控器
- SBUS接收机
- ROS控制板

实验目的

通过ROS控制板解析HT-10A遥控器接收机接收的SBUS协议数据，并通过串口打印各通道的数值。

> 用户需要自己搭建STM32CubeIDE环境！
> 该教程对应的工程源码是【软件资料】文件夹下的SBUS.zip

HT-10A遥控器

需要提前给HT-10A遥控器安装4节5号电池进行供电：

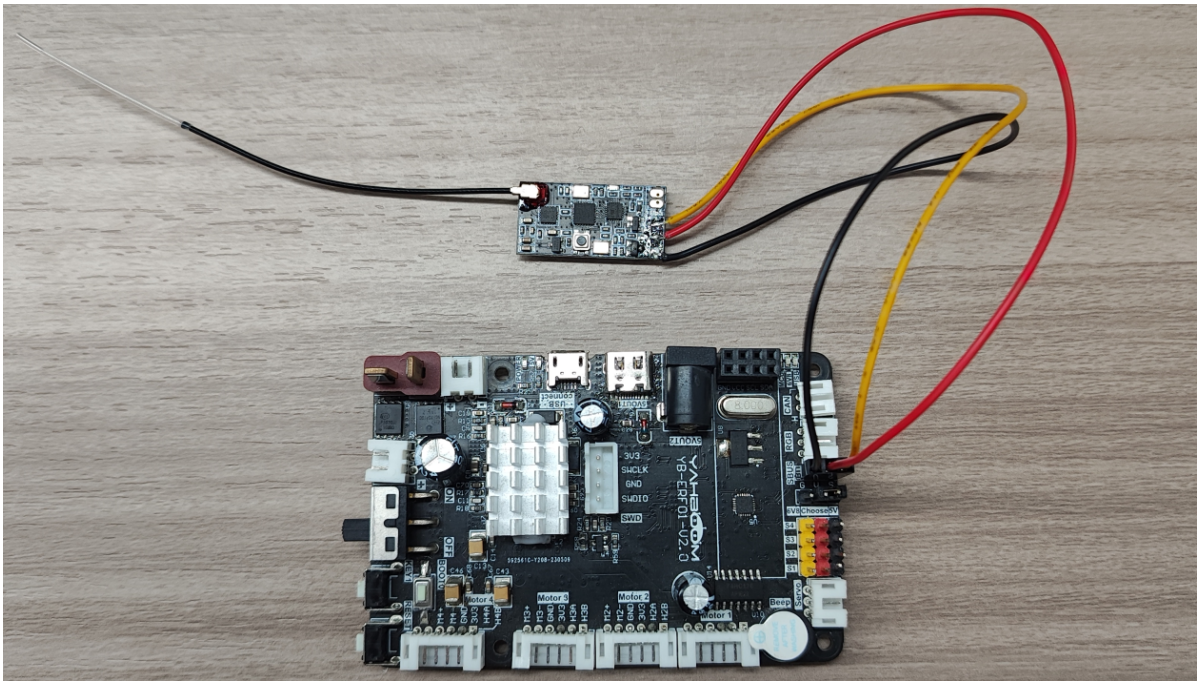


HT-10A遥控器	默认通道：遥控器不进行通道映射
CH1	右摇杆：左右滑动
CH2	右摇杆：上下滑动
CH3	左摇杆：上下滑动
CH4	左摇杆：左右滑动
CH5	SWA通道
CH6	SWB通道
CH7	SWC通道
CH8	SWD通道
CH9	VRA通道
CH10	VRB通道

SBUS接收机

图片演示SBUS接收机与ROS控制板的硬件连接：

SBUS接收机	ROS控制板
S	S
+	V
-	G



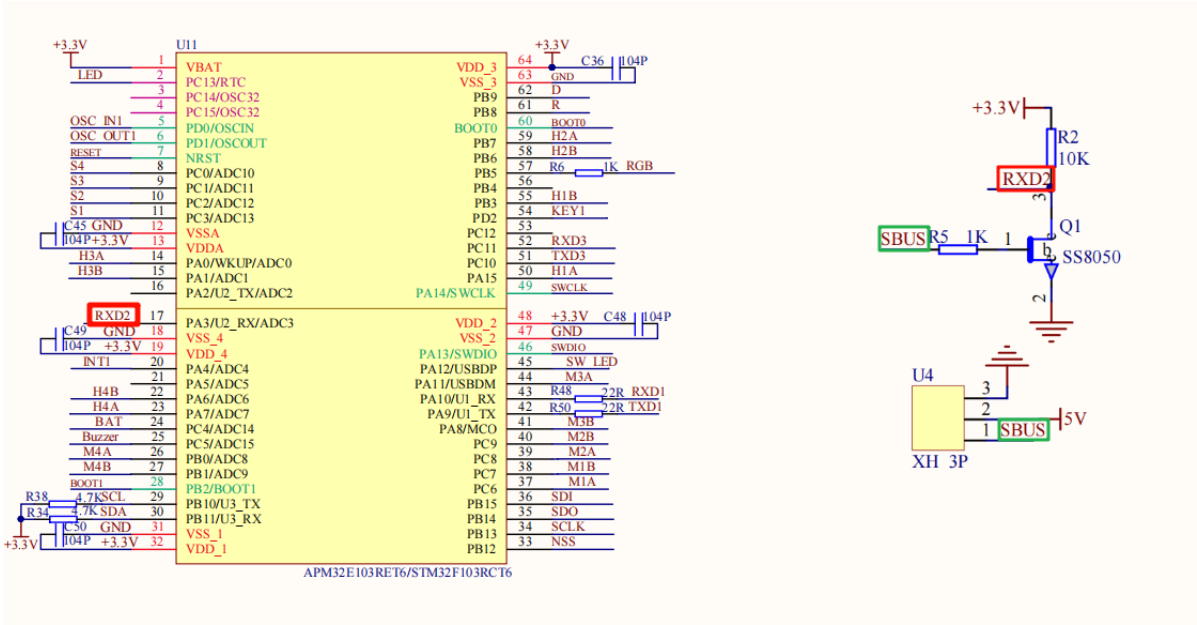
STM32工程

教程只演示主要代码和功能：详细工程代码参考【软件资料】文件夹。

配置引脚

ROS控制板原理图

SBUS接口连接的是ROS控制板的PA3引脚（串口2的RX引脚）：



工程引脚配置

- USART2

Mode：Asynchrinous

Basic Parameters：

Baud Rate：100000

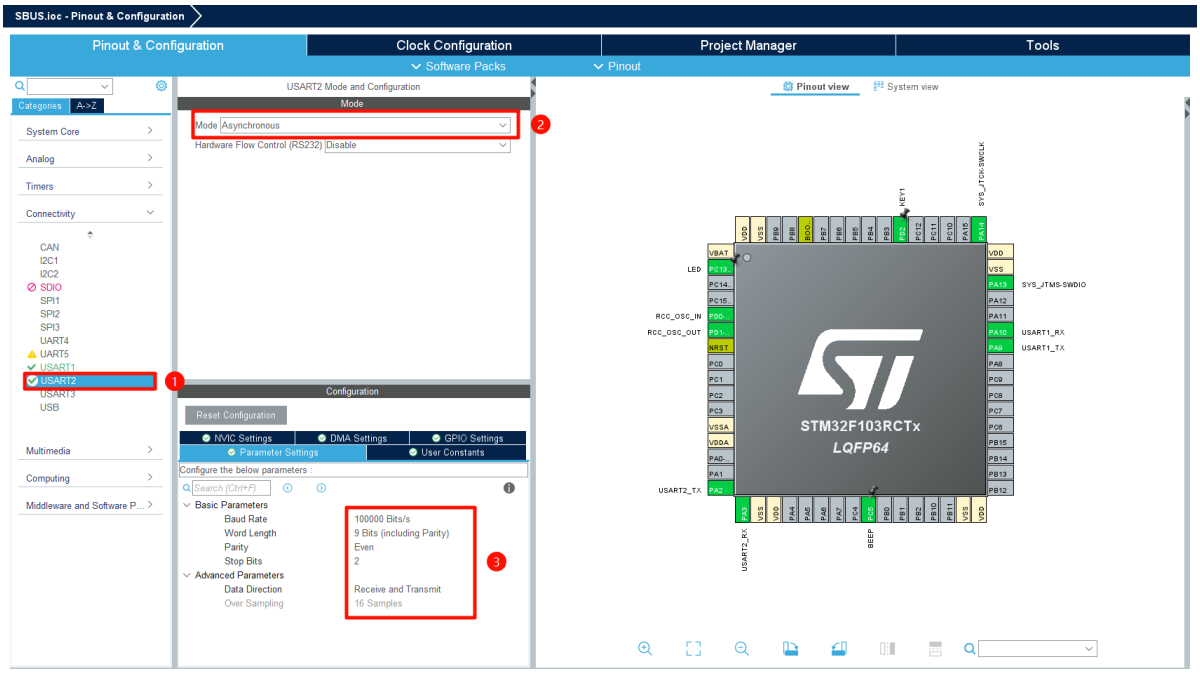
Word Length：9

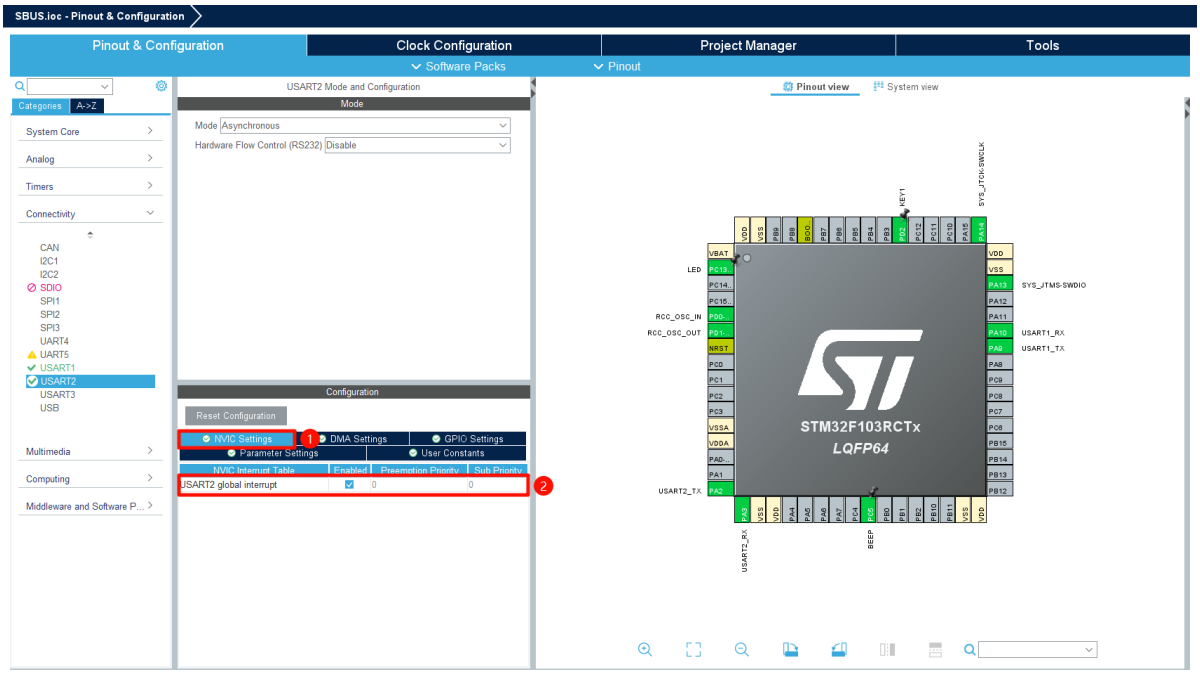
Parity：Even

Stop Bits:2

Data Direction：Receive and Transmit

Enable NVIC：USART2 global interrupt





实现原理

串口2接收SBUS接收机发送的各通道数据并进行解析，将正确的数据通过串口1打印出来。

核心代码

串口功能

- 串口初始化

```
// Initialize USART  初始化串口
void USART_Init(void)
{
    HAL_UART_Receive_IT(&huart1, (uint8_t *)&RxTemp, 1);
    HAL_UART_Receive_IT(&huart2, (uint8_t *)&RxTemp_2, 1);

    printf("start serial\n");
}
```

- 串口中断接收数据

```
// The serial port receiving is interrupted. Procedure  串口接收完成中断
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{

    if (huart == &huart1)
    {
        // 测试发送数据，实际应用中不应该在中断中发送数据
        // Test sending data. In practice, data should not be sent during
interrupts
        USART1_Send_U8(RxTemp);
        // Continue receiving data  继续接收数据
        HAL_UART_Receive_IT(&huart1, (uint8_t *)&RxTemp, 1);
    }
    if (huart == &huart2)
    {
        SBUS_Reveive(RxTemp_2);
        // printf("a:%d\n", RxTemp_2);
        HAL_UART_Receive_IT(&huart2, (uint8_t *)&RxTemp_2, 1);
    }
}
```

SBUS数据

- 接收SBUS数据

```
// Receives SBUS cache data  接收SBUS的缓存数据
void SBUS_Reveive(uint8_t data)
{
    // If the protocol start flag is met, data is received  如果符合协议开始标志，则
开始接收数据
    if (sbus_start == 0 && data == SBUS_START)
    {
        sbus_start = 1;
        sbus_new_cmd = 0;
        sbus_buf_index = 0;
        inBuffer[sbus_buf_index] = data;
        inBuffer[SBUS_RECV_MAX - 1] = 0xff;
    }
    else if (sbus_start)
    {
        sbus_buf_index++;
        inBuffer[sbus_buf_index] = data;
    }

    // Finish receiving a frame of data  完成接收一帧数据
    if (sbus_start & (sbus_buf_index >= (SBUS_RECV_MAX - 1)))
    {
        sbus_start = 0;
        if (inBuffer[SBUS_RECV_MAX - 1] == SBUS_END)
        {
            memcpy(sbus_data, inBuffer, SBUS_RECV_MAX);
            sbus_new_cmd = 1;
        }
```

```
    }
}
```

- 解析SBUS数据

```
// Parses SBUS data into channel values  解析SBUS的数据，转化成通道数值。
static int SBUS_Parse_Data(void)
{
    g_sbus_channels[0]  = ((sbus_data[1] | sbus_data[2] << 8) & 0x07FF);
    g_sbus_channels[1]  = ((sbus_data[2] >> 3 | sbus_data[3] << 5) & 0x07FF);
    g_sbus_channels[2]  = ((sbus_data[3] >> 6 | sbus_data[4] << 2 | sbus_data[5]
<< 10) & 0x07FF);
    g_sbus_channels[3]  = ((sbus_data[5] >> 1 | sbus_data[6] << 7) & 0x07FF);
    g_sbus_channels[4]  = ((sbus_data[6] >> 4 | sbus_data[7] << 4) & 0x07FF);
    g_sbus_channels[5]  = ((sbus_data[7] >> 7 | sbus_data[8] << 1 | sbus_data[9]
<< 9) & 0x07FF);
    g_sbus_channels[6]  = ((sbus_data[9] >> 2 | sbus_data[10] << 6) & 0x07FF);
    g_sbus_channels[7]  = ((sbus_data[10] >> 5 | sbus_data[11] << 3) & 0x07FF);
    g_sbus_channels[8]  = ((sbus_data[12] | sbus_data[13] << 8) & 0x07FF);
    g_sbus_channels[9]  = ((sbus_data[13] >> 3 | sbus_data[14] << 5) & 0x07FF);
    #ifdef ALL_CHANNELS
    g_sbus_channels[10] = ((sbus_data[14] >> 6 | sbus_data[15] << 2 |
sbus_data[16] << 10) & 0x07FF);
    g_sbus_channels[11] = ((sbus_data[16] >> 1 | sbus_data[17] << 7) & 0x07FF);
    g_sbus_channels[12] = ((sbus_data[17] >> 4 | sbus_data[18] << 4) & 0x07FF);
    g_sbus_channels[13] = ((sbus_data[18] >> 7 | sbus_data[19] << 1 |
sbus_data[20] << 9) & 0x07FF);
    g_sbus_channels[14] = ((sbus_data[20] >> 2 | sbus_data[21] << 6) & 0x07FF);
    g_sbus_channels[15] = ((sbus_data[21] >> 5 | sbus_data[22] << 3) & 0x07FF);
    #endif

    // 安全检测，检测是否失联或者数据错误
    // Security detection to check for lost connections or data errors
    failsafe_status = SBUS_SIGNAL_OK;
    if (sbus_data[23] & (1 << 2))
    {
        failsafe_status = SBUS_SIGNAL_LOST;
        printf("SBUS_SIGNAL_LOST\n");
        // lost contact errors  遥控器失联错误
    }
    else if (sbus_data[23] & (1 << 3))
    {
        failsafe_status = SBUS_SIGNAL_FAILSAFE;
        printf("SBUS_SIGNAL_FAILSAFE\n");
        // data loss error  数据丢失错误
    }
    return failsafe_status;
}
```

- 打印SBUS数据

```
void SBUS_Handle(void)
{
    if (sbus_new_cmd)
```

```
    {
        int res = SBUS_Parse_Data();
        sbus_new_cmd = 0;
        if (res) return;
        #if SBUS_ALL_CHANNELS
        printf("%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%d\r\n",
               g_sbus_channels[0], g_sbus_channels[1], g_sbus_channels[2],
               g_sbus_channels[3], g_sbus_channels[4], g_sbus_channels[5],
               g_sbus_channels[6], g_sbus_channels[7], g_sbus_channels[8],
               g_sbus_channels[9], g_sbus_channels[10], g_sbus_channels[11],
               g_sbus_channels[12], g_sbus_channels[13], g_sbus_channels[14],
               g_sbus_channels[15]);
        #else
        printf("%d,%d,%d,%d,%d,%d,%d,%d,%d,%d\r\n",
               g_sbus_channels[0], g_sbus_channels[1], g_sbus_channels[2],
               g_sbus_channels[3], g_sbus_channels[4],g_sbus_channels[5],
               g_sbus_channels[6], g_sbus_channels[7],g_sbus_channels[8],
               g_sbus_channels[9]);
        #endif
    }
}
```

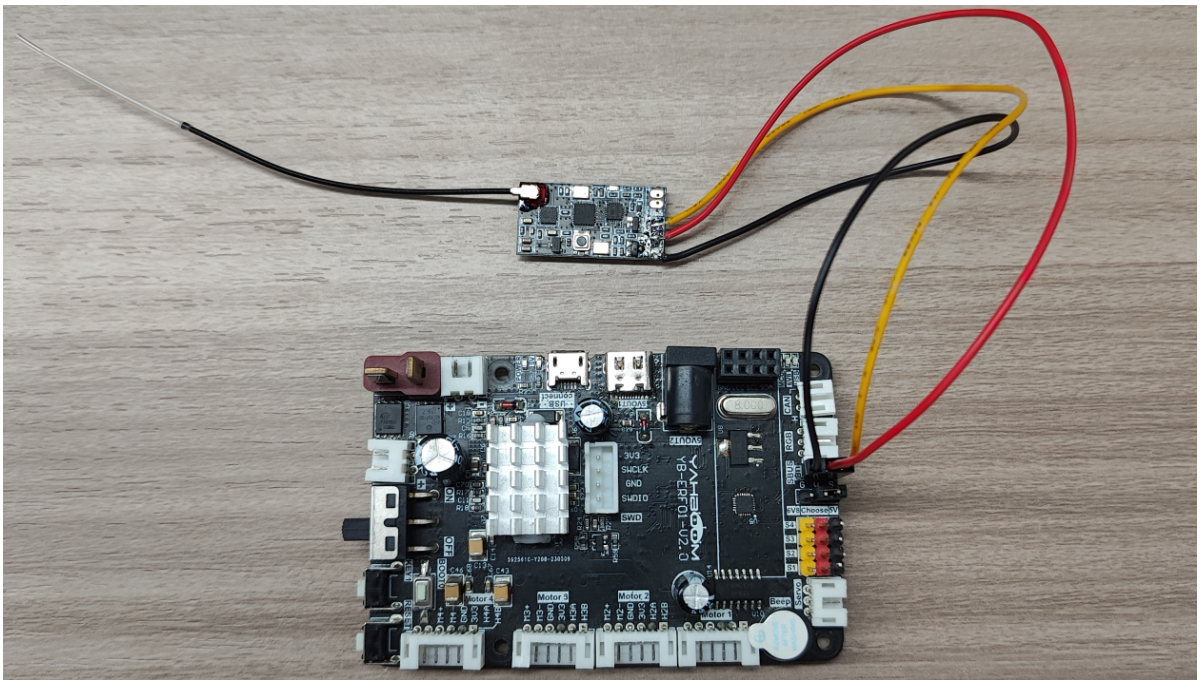
主函数

```
void Bsp_Loop(void)
{
    // Detect button down events    检测按键按下事件
    if (Key1_State(KEY_MODE_ONE_TIME))
    {
        Beep_On_Time(50);
        static int press = 0;
        press++;
        printf("press:%d\n", press);
    }
    SBUS_Handle();

    Bsp_Led_Show_State_Handle();
    // The buzzer automatically shuts down when times out    蜂鸣器超时自动关闭
    Beep_Timeout_Close_Handle();
    HAL_Delay(10);
}
```

硬件连接

SBUS接收机	ROS控制板
S	S
+	V
-	G



下载程序

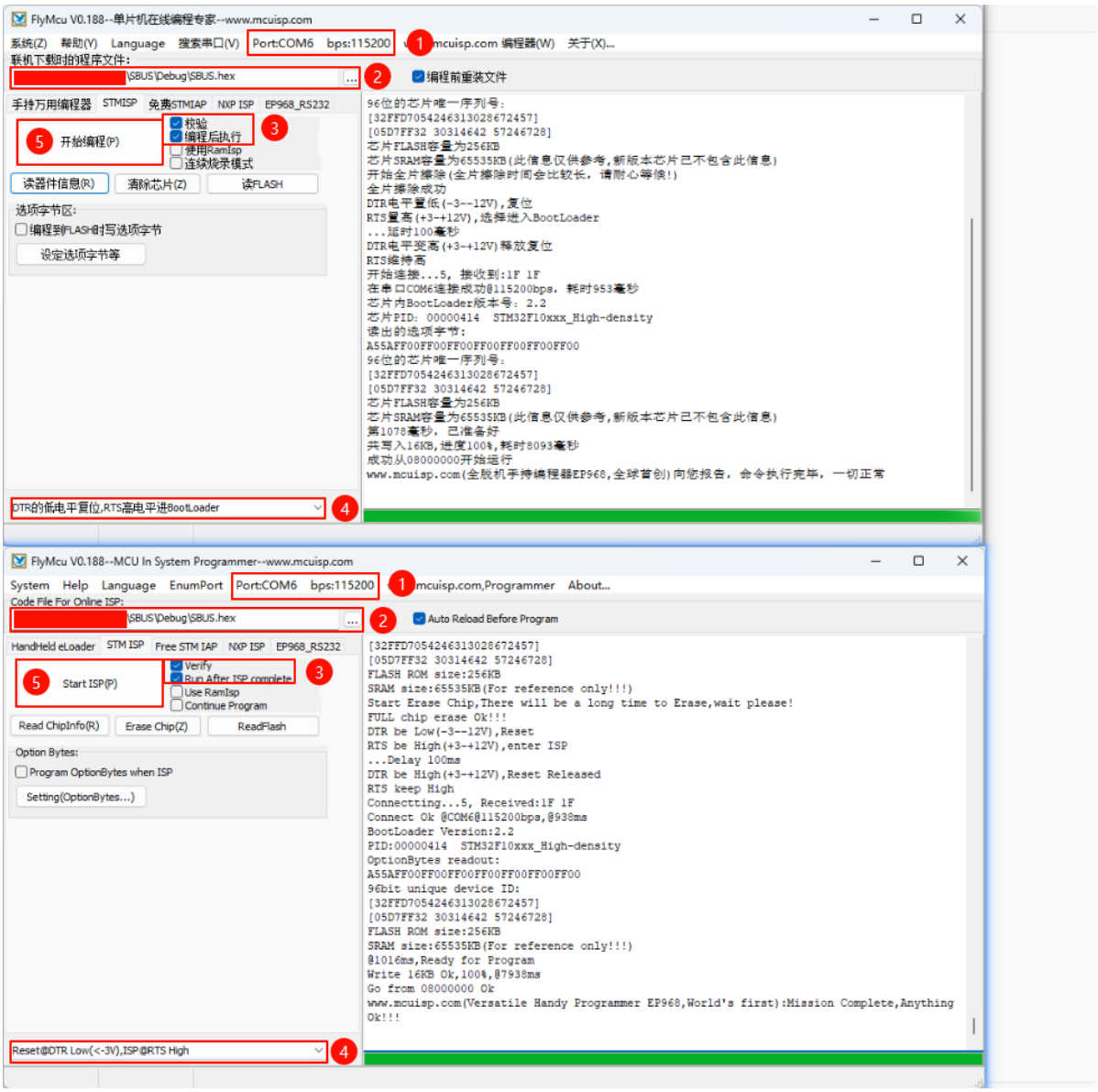
使用FlyMcu程序进行程序的下载：不同开发板进入下载程序模式方法不同，ROS控制板需要再上电前按BOOT按键，连接电脑后松开

①：串口号和波特率设置 → 串口号可以去系统设备管理器查看，波特率选择115200

②：程序文件 → 选择SBUS工程的.hex文件：位于SBUS工程的Debug文件夹下

③④：按照图片中的内容选择

⑤：确认所有选项设置正确，点击烧录选项



实验现象

将该工程的SBUS文件下载到ROS控制板后，LED每隔200毫秒闪烁一次，我们可以通过ROS控制板的MicroUSB接口连接电脑，通过串口查看HT-10A遥控器各个通道的数据。

注意：串口选项设置按照下图设置！

> SBUS接收机与HT-10A遥控器配对可以看【2.SBUS接收机配对】教程，教程有HT-10A遥控器各个通道对应的按键说明

